

This technical report outlines the architectural blueprint for a **Multi-Tenant SaaS Appointment Platform**. The system is engineered to demonstrate robust system design, data integrity, and sophisticated AI integration—core competencies prioritized for Software Engineering (SWE) internship roles.

1. Executive Summary

The platform is a cloud-native, scalable solution enabling businesses to register, configure, and manage client bookings. Utilizing a **multi-tenant architecture**, it ensures strict data isolation while optimizing resource utilization. The engine is specifically designed to mitigate high-concurrency race conditions during peak booking periods and integrates **Large Language Models (LLMs)** to automate customer engagement and provide actionable business analytics.

2. Multi-Tenant Architecture

To support high-density tenant hosting on shared infrastructure, the system employs a **shared database with logical isolation**.

- **Tenant Identification:** Database records are partitioned using a unique `tenant_id` (UUID) to ensure data lineage.
 - **Dynamic Theming:** A configuration engine enables business-specific UI customization. When a client accesses a tenant slug, the frontend dynamically renders layouts and color palettes (e.g., "Sigma" brutalist) based on a retrieved JSON schema.
 - **Security Compliance:** The platform leverages **PostgreSQL Row Level Security (RLS)** to enforce data boundaries at the persistence layer, preventing cross-tenant data leakage.
-

3. Concurrency & Data Integrity

Ensuring data consistency during simultaneous booking attempts is critical. The system implements a hybrid concurrency control strategy to prevent double-booking.

3.1 Synchronous Request Handling

High-stakes operations, such as initial slot reservation, utilize **atomic transactions** and **pessimistic locking**.

- **Implementation:** The backend executes a `SELECT ... FOR UPDATE` query on the specific timeslot row to obtain an exclusive lock.
- **Effectiveness:** This prevents race conditions by forcing concurrent requests to wait for the resolution of the primary transaction, maintaining absolute slot availability accuracy.

3.2 Asynchronous Side Effects

Secondary operations follow an **event-driven architecture** to decouple processes and improve user response times.

- **Implementation:** Post-confirmation, the FastAPI service delegates non-blocking tasks to a background worker pool.
 - **Automated Tasks:** Includes email dispatch, Google Calendar synchronization, and real-time dashboard updates via **WebSockets**.
 - **System Benefit:** This significantly reduces API latency, providing an optimized user experience with near-instant confirmation feedback.
-

4. AI Integration Engine

Advanced LLM integration elevates the platform from a standard booking tool to an intelligent business assistant. This implementation is anchored by a **Self-Hosted Server-Side AI** strategy to maximize cost-efficiency and data governance.

4.1 Self-Hosted LLM for Structured Data Extraction

To facilitate conversational scheduling, the system leverages **TinyLlama (1.1B)** running on the backend infrastructure via **FastAPI** and **Ollama**. This architecture is deliberately chosen to parse complex natural language booking requests (e.g., "I need a consult next Tuesday at 10 AM") into structured JSON entities for database interaction.

This self-hosting model provides dual benefits:

- **Zero Operational Costs:** By eliminating reliance on external LLM APIs, we ensure that the per-call cost to business owners remains zero, providing a powerful, cost-free feature that enhances the platform's value proposition.
- **Data Privacy & Security:** All proprietary tenant data and user inputs are processed entirely server-side, ensuring maximum user privacy and strict data locality without transmission to third-party services.

4.2 Concurrency via Asynchronous LLM Processing

To maintain the responsive user experience required for a high-concurrency SaaS platform, LLM requests are handled asynchronously. The **FastAPI** backend leverages its non-blocking `async/await` paradigms to delegate the TinyLlama parsing tasks to a background worker. This separation ensures that the main API thread remains fully responsive and does not freeze while the AI is performing its computational task, guaranteeing low latency for simultaneous user interactions.

4.3 Broader AI Capabilities

Beyond structured data parsing, the AI engine is used for:

- **Predictive Analytics:** Analyzing customer feedback and booking trends to generate "Trend Reports," helping business owners anticipate shifts in service demand.
 - **Automated Content Generation:** Generating SEO-optimized service descriptions and business profiles for new tenants, accelerating the onboarding process.
-

5. Technical Stack & Schema

The technology stack was selected to prioritize type safety, developer velocity, and horizontal scalability.

5.1 Core Infrastructure

- **Frontend:** Next.js (App Router) with Tailwind CSS for high-performance, responsive interfaces.
- **Backend:** FastAPI (Python), utilizing asynchronous paradigms for efficient I/O handling.
- **Persistence:** PostgreSQL for ACID-compliant transactions and robust relational modeling.
- **Cache Layer:** Redis for distributed locking and session management.

5.2 Entity Relationship Overview

Entity	Primary Attributes	Functional Description
Tenants	id, slug, config	Stores business identities and dynamic configurations.

Entity	Primary Attributes	Functional Description
<i>Services</i>	id, tenant_id, name	Defines catalog items and pricing per tenant.
<i>Slots</i>	id, start_time, locked	Primary availability and inventory management.
<i>Bookings</i>	id, customer, status	Audit-ready records of finalized appointments.
